

8. Designing Class

Example 8.1 (Overview Example)

This example shows **responsibility-driven design**, **low coupling**, **high cohesion**, and **refactoring**.

```
class Order {
    private int quantity;
    private double price;

    public Order(int quantity, double price) {
        this.quantity = quantity;
        this.price = price;
    }

    public double totalPrice() {
        return quantity * price;
    }
}

class OrderPrinter {
    public void print(Order order) {
        System.out.println("Total: " + order.totalPrice());
    }
}
```

- `Order` has **one clear responsibility**: store order data and compute total.
- `OrderPrinter` has **one clear responsibility**: printing.
- Classes depend on each other **through methods only**.
- No duplicated logic.
- Easy to change printing without changing `Order`.

Example 8.2 (Before Refactoring)

```
class Order {
    public int quantity;
    public double price;

    public void print() {
        System.out.println("Total: " + quantity * price);
    }
}
```

- Data and printing logic are mixed.
 - Responsibility is unclear.
 - Harder to change or reuse.
-

8.1 Responsibility-Driven Design

Definition

Responsibility-driven design means each class has clear and small responsibilities.

Features

- One class does one main job
- Responsibilities are explicit
- Classes cooperate by calling methods

Example

```
class Calculator {  
    public int add(int a, int b) {  
        return a + b;  
    }  
}
```

- The class only does calculation.
- No printing or input logic.

Remark

- This design improves readability.
 - This design improves maintainability.
-

8.2 Coupling

Definition

Coupling measures how much one class depends on another class.

Features

- High coupling = many strong dependencies
- Low coupling = few and weak dependencies

Example

```
class A {  
    B b = new B();  
}
```

- A depends directly on B .
- Changes in B may break A .

Remark

- Low coupling is preferred.

- Interfaces help reduce coupling.
-

8.2.1 Implicit Coupling

Definition

Implicit coupling is a hidden dependency not shown by method calls.

Features

- Shared global variables
- Shared static fields
- Shared files or databases

Example

```
class Config {  
    public static int value = 10;  
}
```

- Many classes may depend on `Config.value`.
- Dependency is not explicit.

Remark

- Implicit coupling is dangerous.
 - It is hard to track and debug.
-

8.3 Cohesion

Definition

Cohesion measures how closely related the responsibilities inside a class are.

Features

- High cohesion = methods work on the same data
- Low cohesion = unrelated methods in one class

Example

```
class User {  
    private String name;  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {
```

```
        this.name = name;
    }
}
```

- All methods work on `name` .
- Cohesion is high.

Remark

- High cohesion makes classes easy to understand.
 - High cohesion supports reuse.
-

8.4 Encapsulation

Definition

Encapsulation hides internal data and exposes behavior through methods.

Features

- Fields are usually `private`
- Access through public methods

Example

```
class Account {
    private double balance;

    public double getBalance() {
        return balance;
    }
}
```

- Internal data is protected.
- External code cannot modify directly.

Remark

- Encapsulation reduces coupling.
 - Encapsulation supports responsibility-driven design.
-

8.5 Code Duplication

Definition

Code duplication means the same logic appears in multiple places.

Features

- Same code copied many times
- Bugs must be fixed in many places

Example

```
int total1 = price * quantity;  
int total2 = price * quantity;
```

- Logic is duplicated.

Remark

- Duplication increases maintenance cost.
 - Duplication is a common refactoring target.
-

8.6 Refactoring

Definition

Refactoring means changing code structure without changing behavior.

Features

- Improves design quality
- Keeps program output the same

Example

```
// before  
System.out.println(a * b);  
  
// after  
int total = a * b;  
System.out.println(total);
```

- Behavior is unchanged.
- Code is clearer.

Remark

- Refactoring reduces duplication.
 - Refactoring improves cohesion and coupling.
-

8.6.1 When to Refactor

Definition

Refactoring should be done when code is hard to read or change.

Features

- Long methods
- Large classes
- Repeated code

Remark

- Refactor in small steps.
 - Run tests after refactoring.
-

Chapter 8 Summary (Exam Focus)

- One class → one responsibility
- Prefer **low coupling**
- Prefer **high cohesion**
- Use **encapsulation** to protect data
- Remove **code duplication**
- Use **refactoring** to improve design